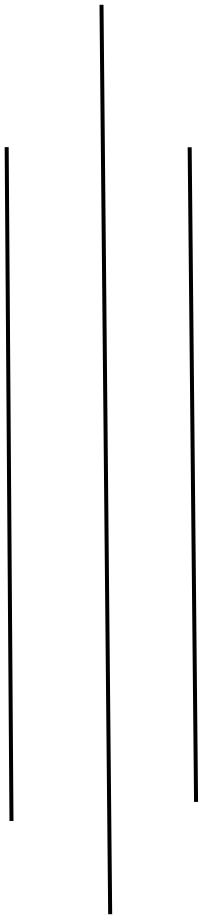
WEEK 5: INTERNSHIP REPORT



Submitted By: Yanji Pakhrin

To

OneCover Nepal

TABLE OF CONTENTS

1. Introduction **1**

2. Objective **1**

3. Scope and Authorization **1**

Practical Task 2

4. Lab Setup 2

i. Platform Used: 2

ii. DVWA URL: 2

iii. Security Level: 2

iv. Testing Environment: 2

v. DVWA Running Status: 2

5. Basic Application Review 3

5.1. DVWA 3

5.2. Vulnerable Module Available 3

5.3. Areas selected for Assessment 4

6. Methodology 5

7. Tools Used: 5

8. Finding 1: SQL Injection (SQLi) 6

9. Finding 2: Reflected Cross-Site Scripting (XSS) 11

10. Challenge Faced 13

11. Key Learning 13

12. Conclusion 13

LIST OF FIGURES

Figure 4.1: DVWA accessed through Web browser....2
Figure 4.2: Security Level set to low....3
Figure 8.1:Apache and MariaDB Services....6
Figure 8.2: Login Page....6
Figure 8.3: Security Level set to Low....7
Figure 8.4: Output of User id 1....7
Figure 8.5: Output After Payload....8
Figure 8.6: Finding Table column using union payload....8
Figure 8.7: Retrieving Database and User name using union....9
Figure 9.1: XSS Module Selected....11
Figure 9.2: Normal Input....11
Figure 9.3: Script Payload....12
Figure 9.4: Pop-up alert confirming payload execution....12

1. Introduction

Damn Vulnerable Web Application (DVWA) is an intentionally vulnerable web application that provides a safe environment for learning and practicing web application security testing.

The assessment was performed in a Kali Linux virtual machine, with DVWA hosted locally. Two vulnerabilities, SQL Injection (SQLi) and Reflected Cross-Site Scripting (XSS) were selected for the assessment using the low security level.

The assessment was mainly focused on how these vulnerabilities work, performing basic tests, recording the observed results.

2. Objective

The main objectives are:

- To perform a basic security assessment of a locally hosted DVWA application.
- To test and understand SQL Injection and Reflected Cross-Site Scripting vulnerabilities.
- To understand the possible security impact of the identified vulnerabilities.
- To provide suitable recommendations for mitigating the identified vulnerabilities.

3. Scope and Authorization

The scope of this security assessment was limited to the locally hosted Damn Vulnerable Web Application (DVWA) running on a Kali Linux virtual machine.

The testing was performed within the authorized lab environment no testing was performed on public websites, real world applications.

Practical Task

4. Lab Setup

i. **Platform Used:**

For the lab setup, Kali Linux was used as the testing environment and Damn Vulnerable Web Application (DVWA) as the vulnerable web application for the security assessment.

ii. **DVWA URL:** The DVWA application was accessed locally through http://localhost/DVWA/

iii. **Security Level:** The DVWA security level was set to low for the assessment.

iv. **Testing Environment:**

The assessment is conducted in an isolated Kali Linux virtual machine. DVWA was hosted locally and is accessible through the localhost address.

v. **DVWA Running Status:**

DVWA was successfully configured and accessed through the web browser at the local DVWA URL.

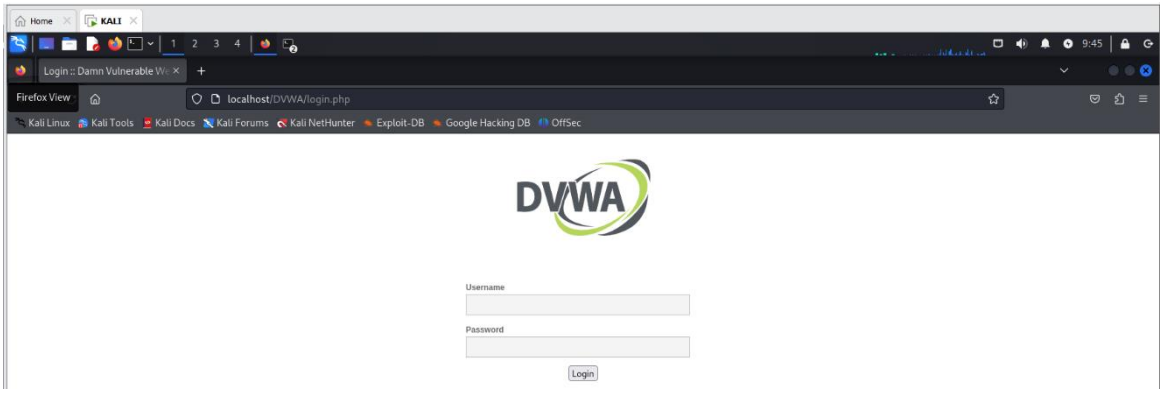


Figure 4.1: DVWA accessed through Web browser

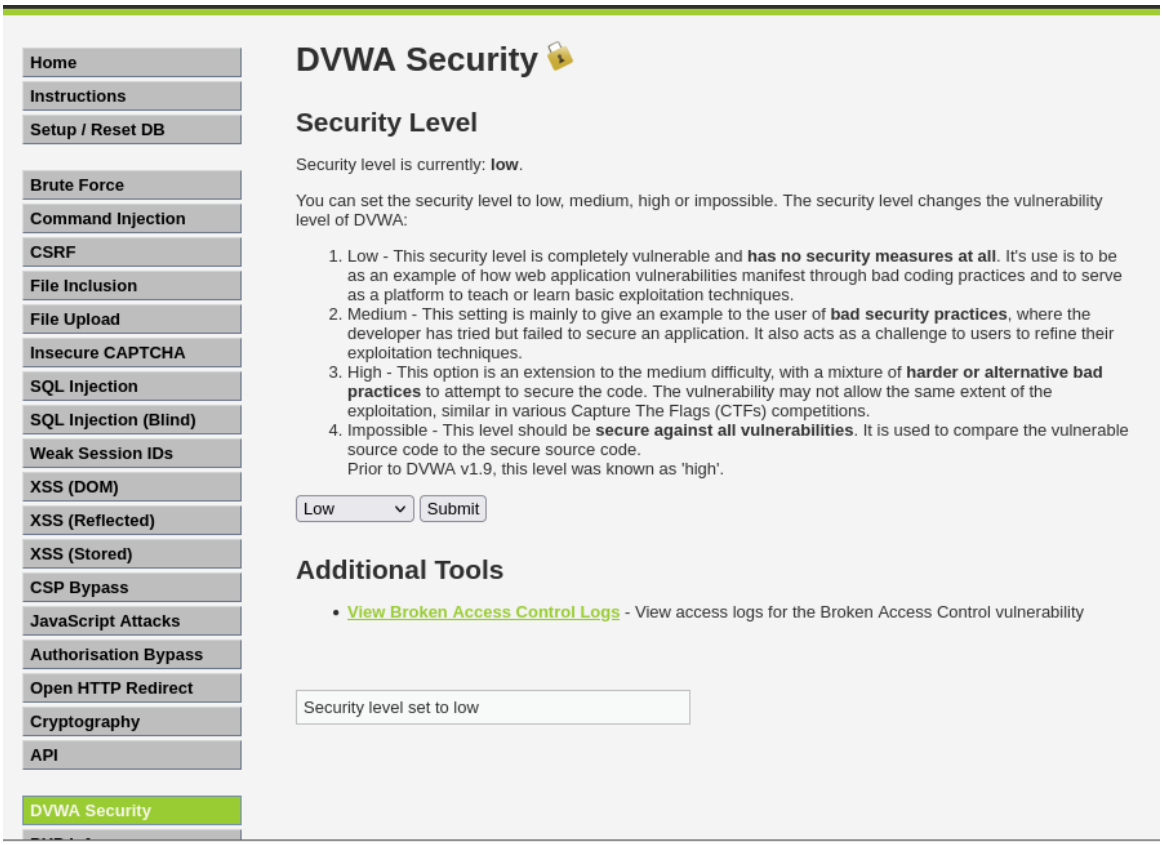


Figure 4.2: Security Level set to low

5. Basic Application Review

5.1. DVWA

Damn Vulnerable Web Application (DVWA) is a PHP and MySQL/MariaDB web application that contains intentional security vulnerabilities for testing and educational purposes.

5.2. Vulnerable Module Available

The vulnerability modules available are:

- Brute Force
- Command Injection
- Cross-Site Request Forgery (CSRF)
- File Inclusion
- File Upload

- Insecure CAPTCHA
- SQL Injection
- SQL Injection (Blind)
- Weak Session IDs
- XSS (DOM)
- XSS (Reflected)
- XSS (Stored)

5.3. Areas selected for Assessment

For this assessment, two vulnerability modules SQL Injection **(SQLi)** and Reflected Cross-Site Scripting **(XSS)** are selected.

i. SQL Injection (SQLi)

The SQL Injection was selected to test how the application handles user input when communicating with the database. The goal was to see if unvalidated input could manipulate database queries and expose sensitive information.

ii. Reflected XSS

Reflected XSS was selected to test how the application handles and displays user input on the webpage. The goal was to check if the application reflects unvalidated input directly in the page allowing malicious scripts to execute.

6. Methodology

Environment Setup: The required system on Kali Linux was set to host and access the DVWA application locally.

Security Configuration and Module Selection: The DVWA security level was configured to low and the SQL Injection and Reflected XSS modules were selected for testing.

Payload Testing: Test Inputs and Payloads were entered into the application input fields.

Observation: Then the observation was done to see how the application processed the unvalidated data.

7. Tools Used:

- Kali Linux
- Apache and MariaDB
- Mozilla Firefox

8. Finding 1: SQL Injection (SQLi)

i. **Vulnerability name:** SQL Injection (SQLi)

ii. **Objective:** The objective was to test the SQL Injection module in DVWA and to check whether user input could change the SQL query, the test also checked if this could show unwanted database results.

iii. **Steps Performed:**

Step 1: Apache and MariaDB services was started in Kali Linux.



Figure 8.1:Apache and MariaDB Services

Step 2: DVWA was opened in Firefox using http://localhost/DVWA/

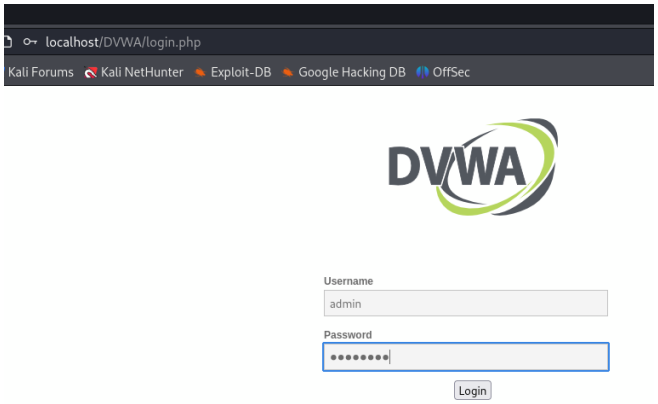


Figure 8.2: Login Page

Step 3: The DVWA security level was set to low.

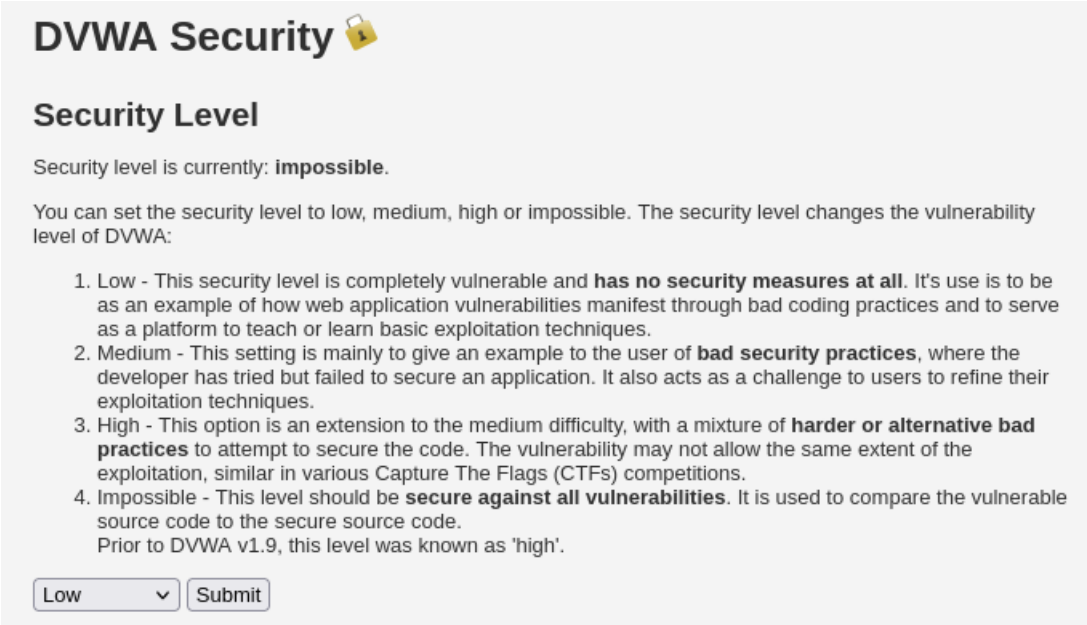


Figure 8.3: Security Level set to Low

Step 4: The SQL injection module was opened.

Step 5: The user ID 1 was entered and submit button was clicked

Step 6: Result is obtained.

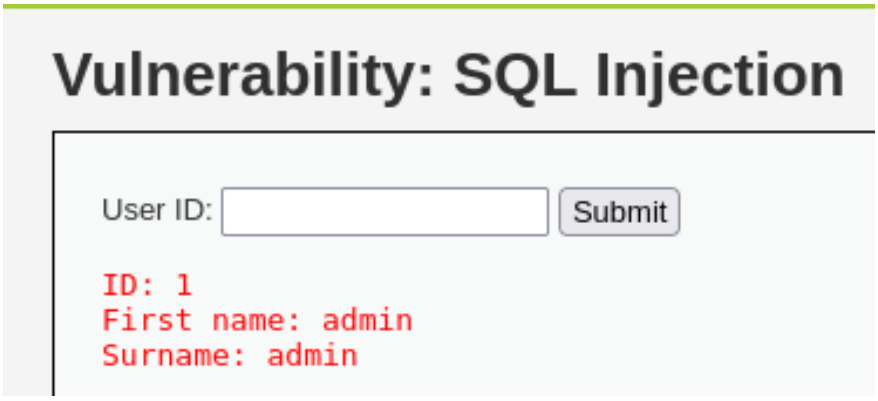


Figure 8.4: Output of User id 1

Step 7: SQL injection payload 1‘OR ‘1’=’1 # was entered and submitted.

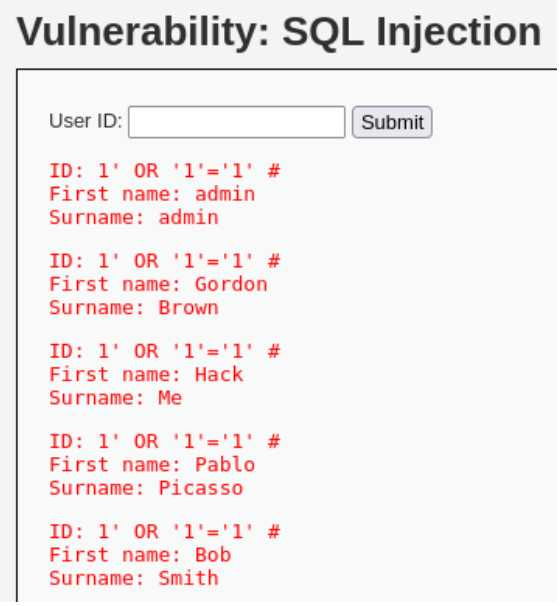


Figure 8.5: Output After Payload

Step 8: The 1’ order by 2# payload was entered to check the number of columns in the database query.

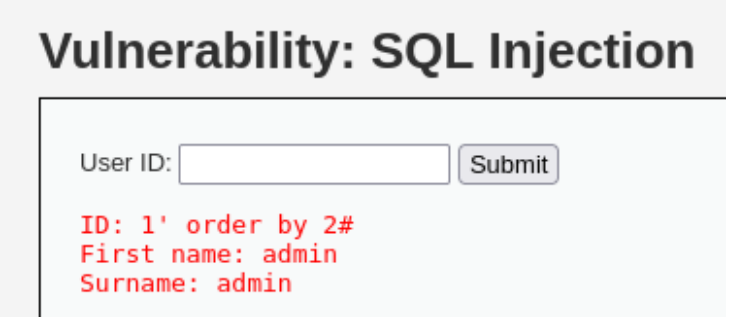


Figure 8.6: Finding Table column using union payload

Step 9: The 1' union select user(), database()# payload was entered and submitted to extract the user name and database name.

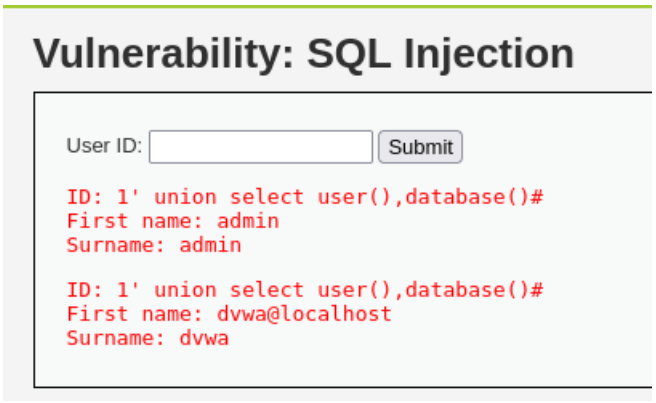


Figure 8.7: Retrieving Database and User name using union

iv. **Result:** The application was not properly sanitize making it vulnerable to SQL, in user input allowing the SQL query to be manipulated.

When 1' OR '1'='1 # was entered, the condition was assumed true. As a result, the application returned all user records.

When 1' order by 2 # was entered, the application accepted the input indicating the user input directly being included in the SQL query which confirmed that the application was vulnerable to SQL injection.

The input 1' UNION SELECT user(), database()# revealed that the application was using the database dvwa@localhost and active database was dvwa.

Overall, these results demonstrates that the application is vulnerable to SQL injection and can expose sensitive information to attackers.

v. **Risk Rating:**

The risk is high because it allows an unauthorized user to manipulate database queries, bypass authentication and extract sensitive information from the database.

vi. Possible Security Impact:

- Unauthorize data access
- Authentication Bypass
- Data Manipulation

vii. Recommended Mitigation

- Using of Parametrized Query
- Input Validation and Sanitation

9. Finding 2: Reflected Cross-Site Scripting (XSS)

i. **Vulnerability name:** Reflected Cross-Site Scripting (XSS)

ii. **Objective:** To test if an attacker could inject malicious script that execute inside a user's browser.

iii. **Steps Performed**

Step 1: The Security level was set Low and Reflected XSS module was selected.

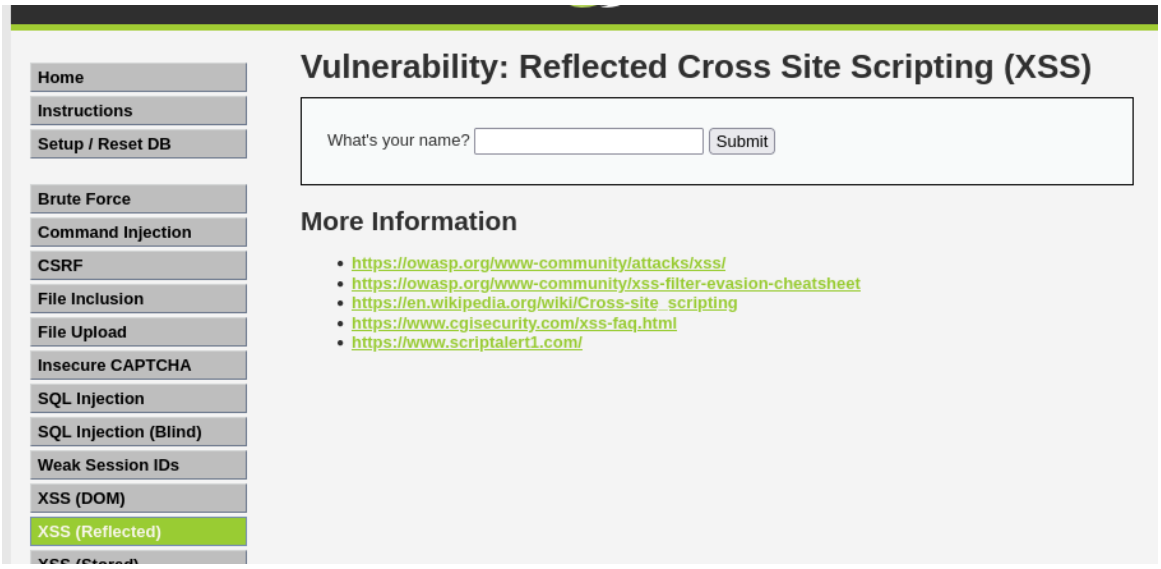


Figure 9.1: XSS Module Selected

Step 2: The input field was filled and submitted to check the normal functioning of the application.

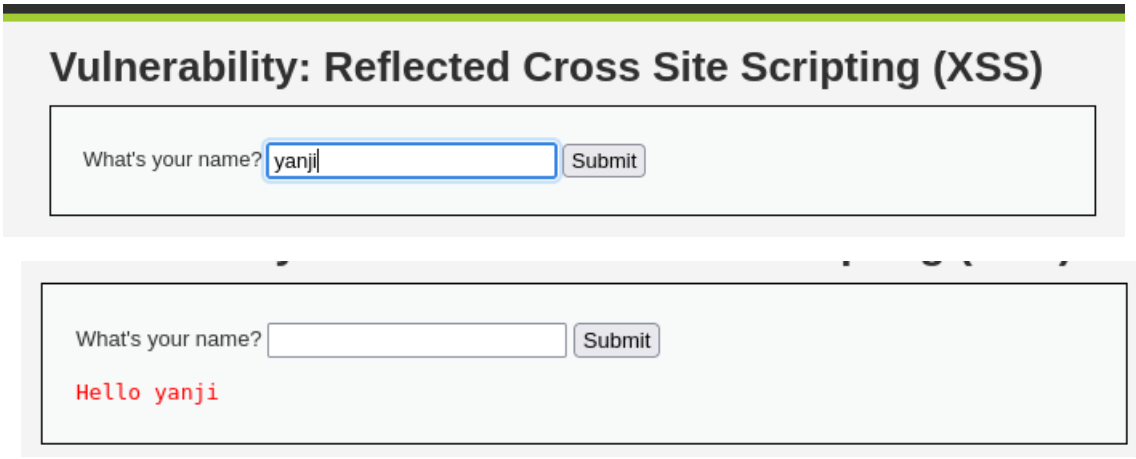


Figure 9.2: Normal Input

Step 3: The script payload <script>alert('Hey')</script> was entered into the input field and submitted.



Figure 9.3: Script Payload

Step 4: The pop-up alert box appeared, confirming that the script executed successfully.



Figure 9.4: Pop-up alert confirming payload execution

iv. **Result:** When normal text was typed, it showed up on the page normally. But when the payload was submitted, the application displayed the text and popped up an alert box in the browser. This shows that the application is vulnerable to Reflected XSS.

v. **Risk Rating**

The risk is medium because an attacker can run malicious scripts in a user's browser but it requires a tricking link to make user into clicking it.

vi. **Security Impact:**

- Session Hijacking
- Credential Harvesting
- Malicious Redirection

vii. Recommended Mitigation

- Input Validation and Sanitization
- Content Security Policy

10. Challenge Faced

- The security level is to be checked frequently because it gets automatically changed to level impossible due to which the payloads are not showing output.

11. Key Learning

- From this lab I learnt the importance of input validation, working of SQL injection and Reflected Cross-Site Scripting (XSS) also ways to prevent those attacks.
- Test should perform on the isolated environment not in real websites or system.

12. Conclusion

Overall, testing SQL injection and Reflected XSS shows how a missing security can allow unauthorized database access or compromise user browsers. Implementing proper input sanitization and secure coding practices will resolve these vulnerabilities.